Design code:

```
// dff_if.sv

interface dff_if (input logic clk);

  logic rst_n;

  logic d;

  logic q;


  modport DUT (

    input  clk, rst_n, d,

    output q

  );


  modport TB (

    input  clk, q,

    output rst_n, d

  );

endinterface


// dff.sv

module dff (dff_if.DUT vif);

  always_ff @(posedge vif.clk or negedge vif.rst_n) begin

    if (!vif.rst_n)

      vif.q <= 0;

    else

      vif.q <= vif.d;

  end
```

```
endmodule
```

UVM:

```
`include "uvm_macros.svh"
import uvm_pkg::*;

// ---------------- Transaction ----------------
class dff_tx extends uvm_sequence_item;
  rand bit d;
  rand bit rst_n;

  `uvm_object_utils(dff_tx)

  function new(string name="dff_tx");
    super.new(name);
  endfunction
endclass

// ---------------- Sequence ----------------
class dff_seq extends uvm_sequence #(dff_tx);
  `uvm_object_utils(dff_seq)

  task body();
    dff_tx tx;
    repeat (10) begin
      tx = dff_tx::type_id::create("tx");
      assert(tx.randomize());
```

```
      start_item(tx);
      finish_item(tx);
    end
  endtask
endclass

// ---------------- Driver ----------------
class dff_driver extends uvm_driver #(dff_tx);
  virtual dff_if.TB vif;

  `uvm_component_utils(dff_driver)

  function void build_phase(uvm_phase phase);
    if (!uvm_config_db#(virtual dff_if.TB)::get(this,"","vif",vif))
      `uvm_fatal("NOVIF","No virtual interface")
  endfunction

  task run_phase(uvm_phase phase);
    dff_tx tx;
    forever begin
      seq_item_port.get_next_item(tx);
      vif.rst_n <= tx.rst_n;
      vif.d     <= tx.d;
      @(posedge vif.clk);
      seq_item_port.item_done();
    end
```

```
  endtask
endclass

// ---------------- Sequencer ----------------
class dff_sequencer extends uvm_sequencer #(dff_tx);
  `uvm_component_utils(dff_sequencer)
endclass

// ---------------- Monitor ----------------
class dff_monitor extends uvm_monitor;
  virtual dff_if.TB vif;
  uvm_analysis_port #(dff_tx) ap;

  `uvm_component_utils(dff_monitor)

  function new(string name, uvm_component parent);
    super.new(name, parent);
    ap = new("ap", this);
  endfunction

  function void build_phase(uvm_phase phase);
    if (!uvm_config_db#(virtual dff_if.TB)::get(this,"","vif",vif))
      `uvm_fatal("NOVIF","No virtual interface")
  endfunction

  task run_phase(uvm_phase phase);
```

```
    dff_tx tx;
    forever begin
      @(posedge vif.clk);
      tx = dff_tx::type_id::create("tx");
      tx.d     = vif.d;
      tx.rst_n = vif.rst_n;
      ap.write(tx);
    end
  endtask
endclass

// ---------------- Scoreboard ----------------
class dff_scoreboard extends uvm_component;
  uvm_analysis_imp #(dff_tx, dff_scoreboard) analysis_export;
  virtual dff_if.TB vif;

  `uvm_component_utils(dff_scoreboard)

  function new(string name, uvm_component parent);
    super.new(name, parent);
    analysis_export = new("analysis_export", this);
  endfunction

  function void build_phase(uvm_phase phase);
    if (!uvm_config_db#(virtual dff_if.TB)::get(this,"","vif",vif))
      `uvm_fatal("NOVIF","No virtual interface")
```

```
  endfunction

  function void write(dff_tx tx);
    if (!tx.rst_n) begin
      if (vif.q !== 0)
        `uvm_error("SCOREBOARD",$sformatf("Reset failed: q=%0b",vif.q))
    end else begin
      if (vif.q !== tx.d)
        `uvm_error("SCOREBOARD",$sformatf("Mismatch: d=%0b q=%0b",tx.d,vif.q))
      else
        `uvm_info("SCOREBOARD",$sformatf("PASS: d=%0b q=%0b",tx.d,vif.q),UVM_LOW)
    end
  endfunction
endclass

// ---------------- Agent ----------------
class dff_agent extends uvm_agent;
  dff_driver    drv;
  dff_sequencer seqr;
  dff_monitor   mon;

  `uvm_component_utils(dff_agent)

  function void build_phase(uvm_phase phase);
    drv  = dff_driver   ::type_id::create("drv", this);
    seqr = dff_sequencer::type_id::create("seqr", this);
```

```
    mon  = dff_monitor  ::type_id::create("mon", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    drv.seq_item_port.connect(seqr.seq_item_export);
  endfunction
endclass

// ---------------- Environment ----------------
class dff_env extends uvm_env;
  dff_agent agent;
  dff_scoreboard sb;

  `uvm_component_utils(dff_env)

  function void build_phase(uvm_phase phase);
    agent = dff_agent::type_id::create("agent", this);
    sb    = dff_scoreboard::type_id::create("sb", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    agent.mon.ap.connect(sb.analysis_export);
  endfunction
endclass

// ---------------- Test ----------------
```

```
class dff_test extends uvm_test;
  dff_env env;
  `uvm_component_utils(dff_test)

  function void build_phase(uvm_phase phase);
    env = dff_env::type_id::create("env", this);
  endfunction

  task run_phase(uvm_phase phase);
    dff_seq seq;
    phase.raise_objection(this);
    seq = dff_seq::type_id::create("seq");
    seq.start(env.agent.seqr);
    phase.drop_objection(this);
  endtask
endclass

// ---------------- Top ----------------
module top;
  logic clk = 0;
  always #5 clk = ~clk;

  dff_if vif(clk);

  dff dut (vif);
```

```
  initial begin
    uvm_config_db#(virtual dff_if.TB)::set(null,"*","vif",vif);
    run_test("dff_test");
  end
endmodule
```